Incident Response Report: ICMP Flood DoS (IR-2026-001)

Oct 23, 2026 · @Chris Giamalis

This is an incident response report for a controlled denial-of-service exercise: a single-source ICMP echo flood against a lab victim host, detected by the SOC pipeline (Suricata to Wazuh, rule 100700, MITRE T1498) and carried through the full response lifecycle from detection to recovery. Because the exercise used a known attacker in an isolated segment, there was no production impact; the value is in exercising and validating the detection-and-response process end to end.

Executive summary

On 26 October 2026, a denial-of-service condition was generated against the victim host (10.20.20.182) from the attacker host (10.20.20.112), both on the attack-range VLAN, as part of a controlled detection-and-response exercise. The attack was a single-source ICMP echo flood. The SOC pipeline detected it automatically: a Suricata rate threshold (sid 1000020) raised an alert once per interval while the flood ran, the Suricata-to-Wazuh pipeline carried those alerts to the manager, and a custom Wazuh rule (100700) escalated each to a level 12 alert mapped to MITRE ATT&CK T1498, Network Denial of Service. The captured flow recorded over 14 million packets and roughly 806 MB directed at the target, confirming a high-volume flood.

The incident was triaged, confirmed as a true positive against two sources of background noise, contained, eradicated, and recovered. Containment was effectively inherent, because the lab's VLAN segmentation bounded the blast radius to the attack range by design. Because the exercise used a known attacker in an isolated segment, there was no production impact and no data at risk. The purpose and the value of the exercise was to validate the full detection-and-response process end to end and to capture the tuning and coverage lessons that came out of it.

Incident overview

Field	Value
Incident ID	IR-2026-001
Date / time detected	2026-10-26 (sensor logs in UTC: 2026-10-27 03:19)
Incident type	Network Denial of Service (controlled adversary-emulation exercise)
Severity	High alert (Wazuh level 12); no production impact (exercise)
Affected system	Victim host, MFF #2 (10.20.20.182), VLAN 20
Source	Attacker host, MFF #1 / Kali (10.20.20.112), VLAN 20
Detection source	Suricata sensor to Wazuh SIEM; rule 100700
ATT&CK technique	T1498 (T1498.001, Direct Network Flood)

Field	Value
Status	Resolved, exercise complete
Analyst	Christian Giamalis

Preparation

The first phase of incident response is everything that exists before an incident, and it is what made this detection and response possible. The relevant controls and telemetry in place were:

- **Network segmentation.** The lab is split into isolated VLANs with a default-block posture. The attack range (VLAN 20) has outbound internet but cannot initiate into the SOC segment (VLAN 30) or the home network. The SOC segment has one-way visibility into the attack range. This segmentation is both a monitoring design and a containment control.
- **Passive network monitoring.** A switch port mirror copies the attacker's switch port to a dedicated capture NIC on the SOC host, feeding a Suricata sensor that sits out of band and does not touch the traffic path.
- **SIEM pipeline.** Suricata writes events to `eve.json`; a Wazuh agent ships them to the Wazuh manager, which decodes, evaluates rules, and surfaces alerts in the dashboard.
- **Custom detections.** A Suricata threshold rule (sid 1000020) rate-counts ICMP floods, and a Wazuh escalation rule (100700) promotes the resulting alert to level 12 and tags it T1498.
- **Known-answer environment.** Because the attacker is a controlled host, every detection can be validated against ground truth, which is what makes the lab a repeatable test bed.

One preparation gap was identified and is carried into the recommendations: Wazuh archives (`logall_json`) were not yet enabled, so non-alerting events are retained only in the sensor's rotating `eve.json` rather than in a durable, searchable store.

Detection and identification

The attack. A single-source ICMP echo flood was generated from the attacker host:

```
sudo hping3 --icmp --flood 10.20.20.182
```

`--icmp` sends ICMP echo requests (type 8), and `--flood` sends them as fast as the interface allows with no waiting for replies, producing a high-rate stream aimed at one target.

Detection. Suricata's threshold rule (sid 1000020, `track by_dst`, 50 packets in 5 seconds) crossed its limit and raised an alert once per interval while the flood ran. The Suricata-to-Wazuh pipeline carried each alert to the manager, where rule 100700 matched it, escalated it to level 12, and tagged it T1498. In the Wazuh Threat Hunting view, filtering on `rule.groups: suricata` showed the burst: repeated level 12 events from the `suricata-sensor` agent, rule 100700, "DoS detected: ICMP flood against target (LAB DoS - ICMP flood to target)," one every few seconds across the flood window.

Raw event. The underlying Suricata alert in `eve.json` captured the flood directly (abbreviated):


```
{
  "timestamp": "2026-10-27T03:22:38.260907+0000",
  "flow_id": 1651713625798976,
```

```
  "in_iface": "eth1",
  "event_type": "alert",
  "src_ip": "10.20.20.112",
  "dest_ip": "10.20.20.182",
  "proto": "ICMP",
  "icmp_type": 8,
  "icmp_code": 0,
  "alert": {
    "signature_id": 1000020,
    "signature": "LAB DoS - ICMP flood to target",
    "category": "Attempted Denial of Service",
    "severity": 2
  },
  "flow": {
    "pkts_toserver": 14090087,
    "pkts_toclient": 13427876,
    "bytes_toserver": 845405220,
    "bytes_toclient": 805672560,
    "start": "2026-10-27T03:19:33.253497+0000"
  }
}
```


The `flow` counters are the evidence of volume: over 14 million packets and about 806 MB directed at the target in a single flow, orders of magnitude above any legitimate ICMP. The sensor's `fast.log` recorded the same detection, one line per 5-second threshold interval:

```
10/27/2026-03:19:33  [1:1000020:1] LAB DoS - ICMP flood to target [Classification:
Attempted Denial of Service] [Priority: 2] {ICMP} 10.20.20.112:8 -> 10.20.20.182:0
10/27/2026-03:19:38  [1:1000020:1] LAB DoS - ICMP flood to target ...
10/27/2026-03:19:43  [1:1000020:1] LAB DoS - ICMP flood to target ...
10/27/2026-03:19:48  [1:1000020:1] LAB DoS - ICMP flood to target ...
```

The sensor logs in UTC, so 03:19 on 27 October corresponds to the evening of 26 October in local time; the Wazuh dashboard renders the same events in local time, which is why the two sources appear to disagree on the date.

Identification and triage. The alert was confirmed as a true positive by separating it from two unrelated event streams that were present during the window, each identified by rule ID, agent or source, and event type:

- Rule 510 "rootcheck" events were a scheduled host-integrity scan running on the sensor's own host, unrelated to network traffic, identified by their rootcheck rule ID and sensor-host source.
- A volume of rule 86601 "STREAM ESTABLISHED packet out of window" events were stream-reassembly artifacts of monitoring through a lossy mirror, not an attack, identified by their decoder-event nature and their correlation with load rather than with a signature.

With those ruled out, the level 12 detection was unambiguous. Post-incident, the raw `eve.json` on the sensor was queried with `jq` to reconstruct context: by event type (`select(.event_type=="alert")`), by the attacker's address (`select(.src_ip=="10.20.20.112")`, which also surfaced the host's benign traffic such as an outbound NTP flow), and by time window. The decisive pivot is `flow_id`: every event for one connection shares it, so `select(.flow_id==1651713625798976)` pulls the complete record of the flood flow. A corroborating attribution signal was present throughout: Suricata repeatedly flagged

`ET INFO Possible Kali Linux hostname in DHCP Request Packet` (sid 2022973) from 10.20.20.112, meaning the attacker host self-identified as Kali through its DHCP hostname.

Containment

In this exercise, containment was largely inherent to the architecture. The flood originated from a single known host inside VLAN 20, and the lab's default-block segmentation means that segment cannot reach the SOC segment or the home network. The blast radius was therefore bounded by design to the attack range, and no containment action was needed to protect anything outside it. The captured evidence confirms this: every DoS record shows source 10.20.20.112 and destination 10.20.20.182, both inside 10.20.20.0/24, so the flood never crossed the segment boundary. This demonstrates that the segmentation built during preparation functioned as a containment control under a live condition, not just on paper.

In a production incident the containment steps would be: identify the source and destination from the alert, preserve the relevant flow records in `eve.json` for analysis before taking disruptive action, then block or rate-limit the source at the gateway with an ACL to drop the flood traffic. For a flood whose source is spoofed or distributed, source-based blocking fails and containment moves upstream to the ISP or a scrubbing service, which is a limitation to acknowledge rather than a step achievable at the lab edge.

Eradication

In this exercise, eradication was simple: the attacking process was stopped at the source by terminating `hping3`. Because the source was a controlled test host rather than a compromised machine, there was no foothold to remove.

In a real incident the distinction matters, and naming it shows an understanding of what eradication actually means. If the flooding host were a compromised internal machine, eradication would require determining how it was compromised, removing the attacker's presence (malware, persistence mechanisms, unauthorized accounts), and closing the entry vector so it cannot be re-enlisted. If the flood were reflected or amplified through third parties, there is no single host to eradicate; the response is upstream filtering rather than host remediation. In short, eradication for a denial-of-service event is only meaningful when the source is something you control; otherwise the work shifts to containment and upstream mitigation.

Recovery

In this exercise, recovery was confirmed once the flood stopped: the victim host returned to normal responsiveness and the alert stream returned to its baseline, with no rule 100700 events after the attacking process ended. No service restoration was required because nothing was taken offline.

In a production incident, recovery is the deliberate return to normal operations with verification at each step: confirm the target's service is restored and performing normally, validate the integrity of the affected system before trusting it again, keep heightened monitoring in place for a defined period to catch recurrence, and only then stand down. The principle is that recovery is not "the attack stopped," it

is "we have verified the environment is healthy and watched long enough to be confident it stays that way."

Lessons learned and recommendations

The lessons-learned phase is where an exercise earns its value. Findings from this incident:

- **Detection thresholds must be tuned to the observation path, not the theoretical rate.** The Suricata threshold was lowered from 500 to 50 packets per 5 seconds because mirror packet loss and the USB 2.0 capture NIC's throughput ceiling meant the higher value never crossed. The captured flow (over 14 million packets) shows how far a real flood sits above the tuned 50-packet trigger. Recorded as a tuning result: before (500, never fired), after (50, fires reliably).
- **Coverage is narrow and should be expanded.** This detection fires only on single-source ICMP echo floods. Recommended next detections, each exercising a different mechanism: SYN flood (TCP state matching), HTTP flood against the web target (behavioral request-rate analysis), and a slow-rate attack such as Slowloris (connection-state monitoring).
- **Enable durable retention.** Turn on Wazuh archives (`logall_json`) so non-alerting events are retained in a searchable store. On the sensor, `eve.json` was already 258 MB with three rotated, gzipped generations and about 519 MB total in `/var/log/suricata`, so the raw context ages out on its own rotation schedule and a later post-mortem becomes impossible once it does.
- **Account for timezone when correlating sources.** The sensor logs in UTC while the Wazuh dashboard renders local time, so the same flood reads as 03:19 UTC in `fast.log` and `eve.json` but as the previous evening in the dashboard. Multi-source correlation has to normalize for this, which reinforces keeping accurate time (NTP) on every host.
- **Keep the attacker-attribution signal.** Suricata repeatedly flagged the attacker's Kali DHCP hostname (`ET INFO`, sid 2022973). Default tool hostnames self-identify an attacker host on the network, which is a free attribution datapoint worth watching for.
- **Reduce triage noise.** The rootcheck events and stream-reassembly events added noise during triage. Recommend filtering rootcheck from the alert view during network testing and tuning stream-event verbosity so the signal is easier to isolate.
- **Document the sensor limitation.** The passive mirror plus USB 2.0 NIC drops packets under load. This is an accepted risk for a lab, worth stating explicitly in the design doc, with a throughput upgrade as a future option if drop counts affect detection.
- **Process validated.** This was the first full end-to-end exercise of the detection-and-response pipeline, confirmed by real captured evidence from attack generation through detection, triage, and recovery.

ATT&CK mapping and technical appendix

Technique: T1498, Network Denial of Service (Tactic: Impact). The technique covers degrading or blocking availability by exhausting network resources, typically through flooding. The precise sub-technique is **T1498.001, Direct Network Flood**, since the flood was sent straight from source to target rather than reflected or amplified.

Attack command (attacker host):

```
sudo hping3 --icmp --flood 10.20.20.182
```

Suricata detection rule (sensor, local.rules):

```
alert icmp any any -> $HOME_NET any (msg:"LAB DoS - ICMP flood to target"; itype:8;
threshold:type both, track by_dst, count 50, seconds 5; classtype:attempted-dos;
sid:1000020; rev:2;)
```

Wazuh escalation rule (manager, local_rules.xml):

```
<group name="suricata,dos,custom,">
  <rule id="100700" level="12">
    <if_sid>86601</if_sid>
    <field name="alert.signature">LAB DoS - ICMP flood</field>
    <description>DoS detected: ICMP flood against target
($(alert.signature))</description>
    <mitre>
      <id>T1498</id>
    </mitre>
  </rule>
</group>
```

Resulting alert: rule 100700, level 12, "DoS detected: ICMP flood against target (LAB DoS - ICMP flood to target)," from the suricata-sensor agent.

Post-mortem queries (sensor, against `/var/log/suricata/eve.json`):

```
jq 'select(.event_type=="alert")' /var/log/suricata/eve.json
jq 'select(.src_ip=="10.20.20.112")' /var/log/suricata/eve.json
jq 'select(.timestamp >= "2026-10-26T23:00" and .timestamp <= "2026-10-27T23:59")'
/var/log/suricata/eve.json
jq 'select(.flow_id==1651713625798976)' /var/log/suricata/eve.json
```